

Assignment 4

Raghav Prabhakar (2881203)
Amea Tran (2731525)
Alicja Dorobis (2746727)

25 May 2026

1 Group Agreement

1.1 Decision making

Our work group is quite structured, and individuals respect and follow the policies of the organization. To maintain that, weekly in-person meetings are necessary to divide the workload and update progress. In addition, some instant calls were made to address the difficulties of each other. However, some points were not satisfied. finished in time as expected. It was caused by the underestimation in workload and the difficulty level of the question. Definitely, it is unavoidable. Hence, the timeline was also flexibly changed and re-allocated the work to guaranty that the assignment is finished in time.

1.2 Group work

From A2, we learned that our group benefits from combining individual strengths with regular group discussion. Some members were more efficient at implementation and debugging, while others contributed more to result interpretation, report writing, and presentation of ideas. We also realized that starting the discussion earlier helps avoid last-minute integration problems. This will help us in A4 because the assignment includes both coding-heavy parts and written linguistic analysis, so we can divide the work more effectively while still reviewing the final answers together.

1.3 Timeline and goal

After understanding the requirement of the assignment, we came up with a timeline as depicted in Table 1

Through the assignment, our main learning goal as a group was to understand how neural models can be used to predict syntactic structure, especially how a transition-based dependency parser builds dependencies through local parsing decisions. By the end of the assignment, we made progress toward this

Date	Responsible member	Task
May 13–14	All members	Read through the assignment, set up the environment, check the starter code, and agree on task division.
May 15–18	Alicja	Work on Part 2 and Part 3: review neural network concepts, complete the MLP notebook, and prepare short written answers.
May 15–21	Raghav	Work on Part 4: implement the transition system, minibatch parsing, neural parser model, and training functions. Run the model and report dev/test UAS.
May 18–22	Amea	Work on Part 5: analyze parser errors, test the garden-path sentence with an online parser, and write the POS-feature analysis.
May 22–23	All members	Share individual parts, check whether the code and written answers are consistent, and discuss unclear results or interpretations.
May 24	All members	Combine the report, proofread explanations, check formatting, and make sure all required output files are included.
May 25	All members	Final check and submission of the zip file and written PDF.

Table 1: Timeline and work plan for assignment.

goal from different perspectives: the implementation part helped us understand the stack-buffer transition mechanism and UAS evaluation, while the error analysis helped us connect parser outputs to linguistic structures such as coordination, prepositional phrase attachment, and garden-path ambiguity. Overall, the assignment helped us see both the technical side of neural dependency parsing and the linguistic reasons why parsing errors occur.

Signatures

Raghav

Amea

Alicja





1.4 Workgroup Reflection

2 Machine Learning & Neural Networks

2.1 Stochastic gradient descent

Stochastic gradient descent, proposed by Robbins and Monro [5], is an optimisation method that approximates the gradient of a loss function by computing the average gradient for one or a subset of components randomly chosen at each step. This means that the direction towards the minimum is not exact but a stochastic approximation. This method is much more computationally efficient than an exact calculation. Therefore, it is well-suited for dependency parsing, where the high-dimensional feature space and large corpus make exact calculations very difficult or almost impossible [6].

2.2 Adam optimizer

2.2.1 Momentum

By using this method, instead of changing its direction of descent after every small iteration, it takes a weighted average of past gradients [3]. So, gradients either reinforce or partially cancel each other out, thus dampening update oscillations. This means that the gradient can descend quicker with smaller oscillations in the direction, like a ball going down a hill and gaining momentum. This low variance helps because it prevents the optimisation algorithm from getting distracted by every small update at each step. Instead, the trajectory of descent gets smoothed out, which means it converges more reliably.

2.2.2 Adaptive learning rate

Adam incorporates an adaptive learning rate for all individual parameters. As $\bar{v}$ is the rolling average of the squared gradients, this means it is large for parameters that receive large gradient updates and small for small updates [3]. Therefore, after dividing the update by $\sqrt{\bar{v}}$, this means that the difference in the updates for each parameter is normalised. This ensures stable and efficient convergence for all parameters by taking a larger step size over flat surfaces (rare gradients) and smaller learning rate over large gradients to avoid overshooting.

2.3 Dropout

2.3.1 Dropout during training

As Srivastava et al. point out, drop out should be applied to avoid overfitting and is applied often as it is a simple method for neural networks [7]. The goal is to create a model that captures data patterns without memorising the noise and generalises well. Dropout regularisation works by randomly deactivating a subset of nodes within layers, creating a “thinned” network. This reduces

co-adaptation and feature bias by ensuring the model does not overly rely on certain nodes. Additionally, this method performs a function of ensemble learning as each iteration trains a different randomised “thinned” network [4].

2.3.2 Dropout during evaluation

During evaluation, dropout should not be applied, as at that moment, we require a full network capacity for accurate prediction. The final model at test time is a combination of the averaged performance of all subnetworks with no dropout [7]. This is achieved by creating one network model where the weights of each node are scaled by the averaged probability being kept activated in the randomised subnetwork, thus creating an approximation from all the thinned networks. Additionally, this averaging method ensures better generalisation for many classification problems.

3 Warm-up: Multilayer Perceptron

For the answers, please refer to the attached notebook.

4 Neural Transition-Based Dependency Parsing

4.1 (a) Transition Table

Stack	Buffer	New Dependency	Transition
[ROOT]	[I, attended, lectures, in, the, NLP, class]	–	Initial Configuration
[ROOT, I]	[attended, lectures, in, the, NLP, class]	–	SHIFT
[ROOT, I, attended]	[lectures, in, the, NLP, class]	–	SHIFT
[ROOT, attended]	[lectures, in, the, NLP, class]	attended → I	LEFT-ARC
[ROOT, attended, lectures]	[in, the, NLP, class]	–	SHIFT
[ROOT, attended]	[in, the, NLP, class]	attended → lectures	RIGHT-ARC
[ROOT, attended, in]	[the, NLP, class]	–	SHIFT
[ROOT, attended, in, the]	[NLP, class]	–	SHIFT
[ROOT, attended, in, the, NLP]	[class]	–	SHIFT
[ROOT, attended, in, the, NLP, class]	[]	–	SHIFT
[ROOT, attended, in, the, class]	[]	class → NLP	LEFT-ARC
[ROOT, attended, in, class]	[]	class → the	LEFT-ARC
[ROOT, attended, class]	[]	class → in	LEFT-ARC
[ROOT, attended]	[]	attended → class	RIGHT-ARC
[ROOT]	[]	ROOT → attended	RIGHT-ARC

4.2 (b) Number of parsing steps

A sentence with n words is parsed in exactly $2n + 1$ steps including ROOT attachment as initial configuration and first step.

Each word is shifted exactly once from the buffer onto the stack, which accounts for n SHIFT operations. In addition, every word (except ROOT) participates in exactly one arc operation (either LEFT-ARC or RIGHT-ARC), which accounts for another n transitions.

Therefore, the total number of transitions required for parsing is linear in the sentence length, i.e., $O(n)$.

4.3 (c) Data

The dataset is provided in the CoNLL dependency parsing format using multiple `.conll` files: `train.conll`, `dev.conll`, and `test.conll`. Each file contains sentences represented token-by-token, where every line corresponds to a single word in the sentence. Sentences are separated by blank lines. The columns are tab-separated, which makes the files easy to parse programmatically.

Each row already contains the linguistic annotations required for dependency parsing. The columns follow the standard CoNLL structure:

1. Token index
2. Word/token
3. Lemma (unused here, represented as `_`)
4. Universal POS tag
5. Language-specific POS tag
6. Morphological features (unused here, `_`)
7. Head index
8. Dependency relation label
9. Additional unused fields

For example, in the row:

```
10 introduced _ VERB VBD _ 0 root _ _
```

the word *introduced* is tagged as a verb (VERB/VBD) and has head index 0, meaning it is the root of the dependency tree. Similarly,

```
11 legislation _ NOUN NN _ 10 dobj _ _
```

indicates that *legislation* depends on the word at position 10 (*introduced*) with the dependency label *dobj* (direct object).

Thus, the data is already fully labeled for supervised dependency parsing. The parser can directly use the head indices and dependency relation labels as training targets.

In addition to the standard dataset files, corresponding files such as `train.gold.conll`, `dev.gold.conll`, and `test.gold.conll` are provided. These contain the gold-standard dependency annotations used for evaluation and comparison against parser predictions. The `.gold.conll` files represent the correct dependency trees and are commonly used to compute metrics such as Unlabeled Attachment Score (UAS).

4.4 (g) Notes on Unlabeled Attachment Score (UAS)

The neural dependency parser achieved a best development-set Unlabeled Attachment Score (UAS) of 88.55% after 10 epochs of training. Using the best model checkpoint selected on the development set, the parser achieved a test UAS of 88.76% on the held-out test set.

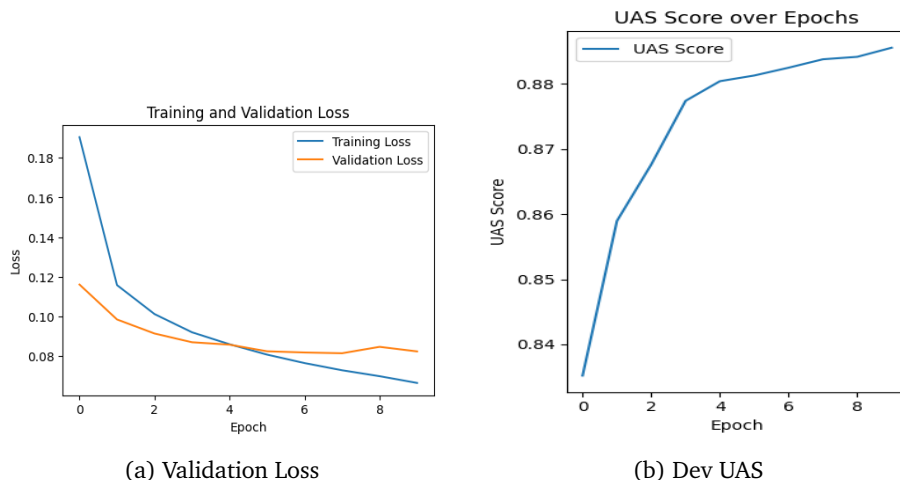


Figure 1: Validation loss and Unlabeled Attachment Score (UAS) over training epochs

Unlabeled Attachment Score (UAS) is a commonly used evaluation metric for dependency parsing that measures the percentage of words whose predicted syntactic head matches the gold-standard head, regardless of the dependency label. UAS is useful because it directly evaluates whether the parser correctly captures the syntactic structure of a sentence. Since dependency parsing primarily focuses on identifying head-dependent relationships between words, UAS provides a clear measure of structural parsing accuracy.

Consider the sentence “She gave the book to her friend.” If the parser correctly identifies “gave” as the root and attaches “She” as its subject, “book” as its direct object, and “friend” as part of the prepositional phrase, then the core syntactic structure is correctly captured. In this case, UAS effectively reflects good performance because the head–dependent relationships are correct even if some dependency labels are imperfect.

Consider the sentence “The man who you saw yesterday left early.” Suppose the parser correctly attaches most words locally, but fails to correctly attach the long-distance dependency between “man” and “left”. Even if many attachments are correct, a few incorrect long-range dependencies can significantly affect the actual syntactic interpretation, but UAS only counts the proportion of correct head attachments and does not distinguish which errors matter more structurally or semantically.

5 Error Analysis

5.1 a

i.

- **Error type:** Verb Phrase Attachment Error
- **Incorrect dependency:** acquisition → citing
- **Correct dependency:** blocked → citing

Semantically, the university is doing the citing and the verbal phrase "*citing concerns...*" explains the reason or consequence for **blocked**; therefore, **citing** should attach to **blocked** instead of **acquisition**.

ii.

- **Error type:** Modifier Attachment Error
- **Incorrect dependency:** had → already
- **Correct dependency:** left → already

The main predicate is **left**, while **had** is only an auxiliary verb that helps form the past tense, so the meaningful action should be modified is **left**.

iii.

- **Error type:** Preposition Phrase Attachment Error
- **Incorrect dependency:** declined → decision
- **Correct dependency:** reason → decision

The parser seems to attach the preposition phrase "**for the government decision**" too high, while there exist reasons explaining these decisions.

iv.

- **Error type:** Coordination Attachment Error
- **Incorrect dependency:** affects → one
- **Correct dependency:** plants → one

Semantically, "**one**" means one plant, so it is the second conjunct coordinated with plants. In universal dependency, the second conjunct usually attaches to the first conjunct rather than to the predicate.

(b)

(i) In Figure 2, the initial parsing of the garden path sentence was computed using **UDPipe**¹, an NLP tool for tokenization, POS tagging and dependency parsing using the Universal Dependencies framework. The parser asserted **raced** as the root of the sentence while **fell** was regarded as adverbial clause modifier.

¹Online demo of UDPipe: <https://lindat.mff.cuni.cz/services/udpipe/>

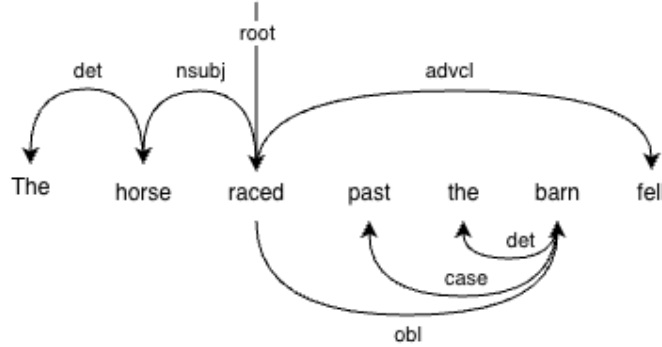


Figure 2: Incorrect dependency parse of the garden-path sentence.

(ii) The correct parsing of the sentence is presented in Figure 3. In interpretation, **fell** is the main verb and the phrase [**raced past the barn**] modifies **horse**. This interpretation is supposed to disambiguate the structure of the sentence with the complex dependency relation.

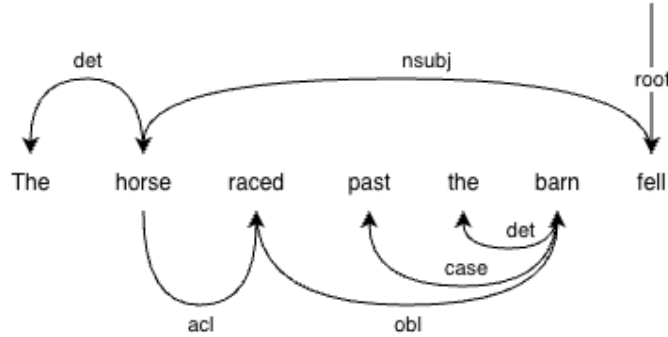


Figure 3: Correct dependency parse of the garden-path sentence.

However, garden-path sentences are challenging for an arc-standard transition-based parser because the parser may commit to an incorrect attachment too early. In this case, one simple way to obtain the correct parse is to prevent the parser from asserting RIGHTARC to create dependency $\text{horse} \leftarrow \text{raced}$. This solution is seemingly impractical with respect to the characteristics of the algorithm, since the parser makes local decisions one at a time. In addition, when decisions are made, the procedure may not be reversed. Thus, Dary et al. proposed adding backtracking to transition-based parsing and further explore other solutions in cases where new findings contradict the current solution. Regarding our case, when the parser reaches **fell**, it realizes that the token should be the main verb. It can return to the earlier parser state, avoid creating the dependency $\text{horse} \leftarrow \text{raced}$, and instead build the correct dependency fell

→ horse, while treating raced past the barn as a modifier of horse. The solution helps the parser recover from earlier wrong decisions, especially in garden-path sentences.

(c)

One hypothesis of why part-of-speech tags are important for dependency parsing is that they provide grammatically category information that helps the parser decide which attachments are likely. Chen and Manning utilized POS tags, along with word form and dependency-label features, as dense representations in a neural transition-based dependency parser. The result of the model probing showed that the perceptrons that had a direct connection to the POS feature are most predictive in the parsing, highlighting the importance of the part-of-speech feature in dependency parsing.

Therefore, we implemented a behaviour analysis on the trained model from the previous section to investigate the validity of the hypothesis. In Figure 4, we analysed the dependency parsing of sentence *"They can fish ."*. The main difficulty arises from lexical ambiguity: the forms **can** and **fish** are lexically ambiguous. The word **can** could be a modal auxiliary, as in the sentence, or a noun, as in example *"a can of fish"*. Similarly, **fish** could be a verb or a noun. In the parse shown, the POS tags resolve this ambiguity. **Can** is tagged as AUX, and **fish** as VERB. This makes **fish** being regarded as the root of the sentence, with **They** attached as its subject and **can** attached as an auxiliary. Therefore, the sentence is interpreted correctly as its original intent *"They are able to fish."*

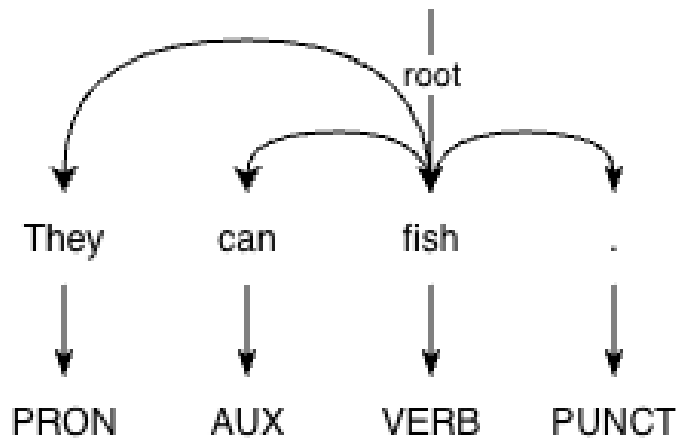


Figure 4: The parsing in sentence *"They can fish ."*

Moreover, POS tags also benefit dependency parsing in other syntactic relation attachment. In the sentence *"Many managers and traders had already left."*, the parser faces two important structural decisions. First, the tag CCONJ for **and** signals that **managers** and **traders** form a coordination structure. Hence,

traders was attached to **managers** rather than independently attached to the verb, as presented in Figure 5. Second, the tags AUX, ADV, and VERB help distinguish the verbal phrase *had already left*. In other words, *had* functions as an auxiliary, *already* modifies the main event, and *left* is the main verb. This example shows that POS features help the parser identify both coordination and verbal structure, leading to more accurate attachment decisions.

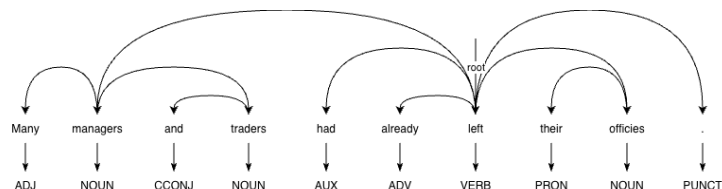


Figure 5: The parsing in sentence "*Many manager and traders had already left their offices .*".

However, the behaviour of the parser is not always as expected. In the sentence "*She looked up the answer .*" (as depicted in Figure 6, *looked up* functions as a phrasal verb meaning "searched for", rather than as a verb followed by a prepositional phrase. Therefore, **up** should ideally be tagged as PART, not ADP as in the parsing. It may lead the parser to treat *up the answer* as a prepositional phrase, instead of recognizing **up** as part of the verbal expression *looked up*. This shows that POS-tag accuracy can directly affect dependency parsing accuracy. Specifically, when the POS tag gives the wrong syntactic cue, the parser is more likely to build the wrong attachment structure.

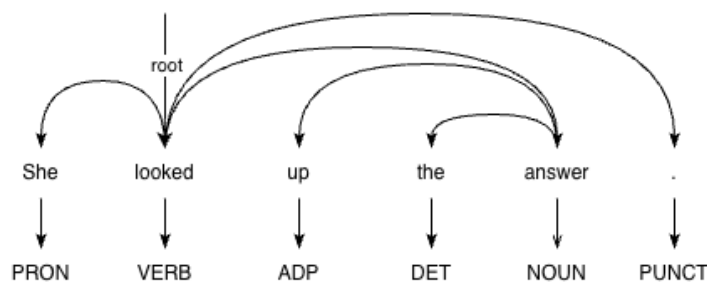


Figure 6: The parsing in sentence "*She looked up the answer .*".

References

- [1] Danqi Chen and Christopher D Manning. A fast and accurate dependency parser using neural networks. In *Proceedings of the 2014 conference on em-*

- pirical methods in natural language processing (EMNLP)*, pages 740–750, 2014.
- [2] Franck Dary, Maxime Petit, and Alexis Nasr. Dependency parsing with backtracking using deep reinforcement learning. *Transactions of the Association for Computational Linguistics*, 10:888–903, 2022.
 - [3] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2017. doi: 10.48550/arXiv.1412.6980. URL <https://doi.org/10.48550/arXiv.1412.6980>.
 - [4] Asad Pathan. Dropout regularization in deep neural networks. URL <https://medium.com/@pathanasad3406/dropout-regularization-in-deep-neural-networks-a075952eae36>.
 - [5] Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951. doi: 10.1214/aoms/1177729586. URL <https://doi.org/10.1214/aoms/1177729586>.
 - [6] Ehsan Shareghi, Yitong Li, Yujia Zhu, Roi Reichart, and Anna Korhonen. Bayesian learning for neural dependency parsing. In Jill Burstein, Christine Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 3509–3519. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1354. URL <https://doi.org/10.18653/v1/N19-1354>.
 - [7] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.